

MAKER (COUNTER)CULTURE

RONI BANDINI



Maker (counter)Culture

The original text was written in Spanish. This version was translated with the assistance of AI and may contain errors or inaccuracies.

Roni Bandini @ronibandini

Buenos Aires, Argentina

CC BY 4.0 License

<https://creativecommons.org/licenses/by/4.0/>

This work is offered under a Creative Commons Attribution 4.0 International (CC BY 4.0) license. You may share, copy, adapt, and reuse its contents, including for commercial purposes, provided that appropriate attribution is given and changes are indicated.

First version 10/2024

Second version 12/2025

Third version 10/2026

INTRODUCTION

*"It is inevitable that every culture must negotiate with technology, either
intelligently or otherwise"*

Neil Postman

"Some men know how to build magical machines"

Alberto Laiseca

There is a perspective that would rather have us tired, submissive in conformity, and living in an algorithmic underworld where our joys, our economy, even our relationships are being digitized.

What role do the devices we use play in this scheme of domination and control? How functional are their labyrinths of use? And above all, what starts to happen when we move from unthinking consumers to creators?

This book is about a path where technology, counterculture, art, and activism converge. A path that often feels like a detour. A path where the disturbance of making machines and devices that perhaps should not exist has the power to open new conversations and question the sacredness and absurdity of what has been established.

This book is about Maker Counterculture

Roni Bandini

MACHINES, AUTOMATA, ROBOTS, AND CYBORGS

*"Machines articulate themselves and define themselves against the disorder
of organic nature"*

Anonymous

In Maker Counterculture we build hardware. Why hardware and not software? In recent decades there has been an invitation, almost an imperative, to develop software. This invitation came with promises of possibilities, free will, and success. But what is the agenda and subtext behind this invitation? To begin with, we need to dismantle the idea of possibilities: software is mostly harmless when confined to a sandbox. That is, by controlling the hardware, any software has a limit. The invitation to develop software and software alone covertly imposes the battleground. On what terrain are we going to choose to fight our battles? On the terrain suggested by the hegemon? Or on one where, at least for now, we have some chance?

On the other hand, hardware and the manipulation of physical objects in general offer something that the symbolic work of software cannot. If we cut a cable, current no longer flows. If the screw does not fit, it does not fit. In some way hardware is a foothold in times of post-truth. At some point, every maker discovers that a machine, any machine, is difficult to ignore; its mere existence confronts its surroundings, demanding that we use it, move it, work around it, switch it off, or destroy it. Ultimately we make machines to make clear that we are paying attention to the subtext; we make OUR machines – open, messy, incorrect, defensive, and subversive – to balance the OTHER machines – closed, correct, dehumanized, lethal – and we make machines, ultimately, in an attempt to understand our own essence.

But what are we talking about when we talk about machines?

MACHINES

One possible definition of a machine is the following:

“A manufactured object composed of a set of parts used to facilitate a particular task, generally by transforming or harnessing a form of energy”

It is obvious that a sewing machine is a machine. Its name says so, and it transforms kinetic force into sewing fabric, for example. But the definition can have subtler, human, and philosophical implications: in ancient Rome, for example, slaves were called “natural machines,” and architect Le Corbusier said that a house was a “machine for living in.”

Machines, curiously, do not have to work: they may not work, they may operate erratically, they may have fictional principles, they can even be conceptual, artistic, or shut themselves off. Even so, they remain machines.

USELESS MACHINES

This concept was coined by the Italian artist Bruno Munari while developing mobiles. At first he wanted to make a work of art that interacted with its surroundings. He did it almost as a joke in response to the Futurist movement. Munari called them “machines” because his mobiles were made of moving parts, and added “without purpose” because they did not produce goods for

material consumption, reduce manual labor, or increase capital.

Perhaps it is useful to think of the machine as a perspective or an analysis of the harnessing of forces that can be applied to both an object and a subject.

DUAL RELATIONSHIP

The relationship between machines and humans has always been marked by a certain suspicion and fear. The Luddites were nineteenth-century English workers who sabotaged industrial looms and spinning machines that threatened their livelihoods. Samuel Butler said in 1872 that every non-essential machine should be destroyed. Theodore Kaczynski, a prominent Berkeley mathematician, went to live in a cabin in 1978 without electricity or running water and began attacking those he saw as responsible for technological progress. Ten years before the Luddites' sabotage actions, the Jacquard loom had been invented, a revolutionary machine that used punched cards to weave patterns into cloth. This machine is one of the oldest antecedents of computing and the use of binary databases. From one point of view, then, the Luddites may not have been destroying machines merely to preserve their jobs, but rather initiating a kind of prophetic resistance to a future in which machines would extend their tyrannical domination over human beings.

The Luddites took their name from Ned Ludd, an emblematic figure who

sabotaged the first looms. However, so far no conclusive evidence of his existence has been found. As if he had only passed through history to reveal disturbing echoes of the future.

At the same time, we can go back into the past and find some quite opposite references concerning the human-machine relationship. In 1909 the Italian Futurist movement suggested that the concept of beauty was shifting from nature and women toward the machine. The machine, despite its latent threat, is the ultimate evidence of human skill. Technological chauvinism and the status of technologists in today's society confirm that beauty still resides in the machine.

AUTOMATA

One possible definition of an automaton is the following: A machine that imitates the form and movements of a living being.

Timeline

1000 BC

In the Bible, the Book of Psalms mentions the Golem, perhaps the first automaton. It was a clay creature that was difficult to control. Certain Golems were animated using words. The word EMET (truth) activated it and MET

(death) deactivated it.

300 BC

In Greek mythology, Talos was a bronze automaton that circled Crete three times a day to protect it from invaders.

1100 AD

Ismael Al Jazari published instructions for an automaton that served drinks. Every seven and a half minutes, the doors of a cabinet opened to reveal an automaton of a slave girl holding a full wine cup and a towel. The king served himself wine, wiped his mouth, and the automaton disappeared into the cabinet.

1495

Da Vinci's Armored Knight. It moved its limbs and had the unusual feature of allowing its movements to be programmed with pegs to alternate sequences.

ROBOTS

The automaton is an ancestor of the robot, since the term robot was coined by the Czech writer Karel Čapek in a 1920 play. In the play, robots were made from organic matter, so the word later acquired another dimension. In Czech,

ROBOTA means forced labor. The word derives from “slave.”

A question worth asking is: why are robots usually anthropomorphic? On the one hand, humans are the measure of everything, but there may be other reasons as well. As E. Muslin said in the book *Máquinas del Siglo XX*: “An elegantly made mechanical doll provokes a careful attitude; no one will kick it, and no one will enjoy hitting it, say, with a hammer.”

CARDINALITY

We can then identify a line.

1. The machine is the harnessing of forces to facilitate a task.
2. The automaton is a machine that imitates the movements of living beings.
3. The robot is an automaton – often anthropomorphic – forced to do our work by force.

Thinking about this cardinality when we build our machines leads to better associations and greater depth.

CYBORGS

The term cyborg comes from the contraction of cybernetic organism –“cybernetic organism”– and was coined in 1960 by Manfred E. Clynes and Nathan S. Kline in the article *Cyborgs and Space*, where they imagined the technological modification of the human body to allow autonomous adaptation to the extreme conditions of space. A cyborg integrates organic and technological components into a single system regulated through information, control, and feedback. Just as the automaton imitates life and the robot performs slave labor, the cyborg erases the boundary between body and machine.

There is no need to imagine a being half human and half machine: a person with a prosthesis or a pacemaker can be considered a cyborg

MAKER CULTURE

*"If you really want to understand something,
then what you have to do is build it"*

Simon Schaffer

Let's start with an obvious point: a maker is someone who defines themselves by what they make rather than by what they buy or use, for example.

Maker Culture takes place at the intersection of the DIY movement – do it yourself – and the branch of hacker culture inclined toward hardware. Here, hacker is understood in its original sense: a computer enthusiast who makes computers work in a different and personal way.

PRINCIPLE

Maker Culture proposes a fundamental principle, very elegant, in the form of a rhetorical question: Why trust devices when they present us with frustrating labyrinths of use, spy on us, prevent us from using their capabilities to the fullest, and demand replacement before their time?

MOTIVATIONS

Maker Culture can be motivated by necessity. In the 1990s, the Cuban publisher Verde Olivo compiled people's experiences repairing and building devices from household appliances and parts available on the island in a

manual titled “Con nuestros propios esfuerzos”. In Argentina, bureaucracy and protectionism made it difficult to import electronics, so ingenuity was necessary to adapt what already existed through creative processes.

Sometimes Maker Culture can also be motivated by philosophy. In developed countries, without shortages or prohibitive costs, there are makers who choose to build even when it is more expensive and complex, as a reaction to consumerism. And this happens everywhere when one adopts a maker mindset. That is, when one stops being an unthinking entity that meekly accepts whatever device is placed in front of them and assumes the role of creator.

KEY ASPECTS

DIY (Do It Yourself)

The practice of doing things yourself instead of buying or outsourcing them, even when the process is more complex or more expensive.

Community and Collaboration

The maker constantly relies on collective knowledge and practices. There is a strong ethic of sharing projects, taking the time to write tutorials, comment



code, and offer assistance. This does not mean a maker must always open all their projects. Keeping one or more projects closed for contractual reasons or personal choice does not exclude them from the community.

Active Learning

Prioritizing constructionism, experimentation, and iterative design (prototyping, failing, correcting, and improving) as the main way to acquire knowledge and skills.

Interdisciplinarity

Maker Culture erases the boundaries between disciplines by combining traditionally separate skills, such as electronics, programming, design, crafts, and mechanics, in the creation of devices.

MAKER VERSUS INVENTOR

The figure of the inventor is interesting and does not exclude other talents. Roberto Arlt, a great Argentine writer, invented unbreakable stockings. Lewis Carroll, author of *Alice in Wonderland*, developed an ingenious system for writing in the dark. Actress Hedy Lamarr invented Spread Spectrum, which later enabled WiFi. However, the inventor differs substantially in that they operate from premises focused on patents and commercialization. The maker,

by contrast, “makes” without needing to inaugurate something or close off the possibility for others to create something similar or even exactly the same.

THE TERM MAKER

The term maker emerged around 2005, when several events coincided: The Arduino board was released that year, developed by Massimo Banzi and the team at the Interaction Design Institute Ivrea. At the same time, Make magazine, founded by Dale Dougherty, was launched. A year later, the first Maker Faire was organized in San Mateo, California, and Adrian Bowyer launched the RepRap 3D printing project.

Nevertheless, makers have always existed, just without that label. To name a few: Juan Baigorri Velar in Argentina with his 1938 mineral-detection machine, Don Lancaster in the United States with the 1973 TV Typewriter, and Clive Sinclair in England with the 1977 Sinclair MK14.

As for publications, the Argentine magazine *Lúpin* from 1966 was a maker publication, as were the American *Popular Electronics* from 1954, the British *Everyday Electronics* from 1971, and the Dutch *Elektor* from 1961.

MAKER COUNTERCULTURE

*"What is needed are people who can become stranger
than the strange world we have produced"*

John Durham Peters

Counterculture is opposition to and questioning of prevailing social norms and values. It is often associated with the United States of the 1960s, but it also applies to other similar circumstances, such as the French Bohemia of the 1600s. Counterculture entails rejection of and criticism toward what is established, which in technological terms and with the inclusion of Artificial Intelligence, takes on new scales and meanings.

The term Counterculture, so prominent during the 1960s and 1970s, seems to have been devalued. In certain academic circles it was argued that after postmodernity there can be nothing outside culture or against culture, and while this makes sense for an essay, it is a somewhat sinister perspective insofar as it tries to establish that no struggle is possible.

Many of us believe that Counterculture is still possible and, not only that, that it is more necessary than ever.

CONSTRUCTION

Maker Counterculture is built on Maker Culture. It shares the same principles, dynamics, and protocols, but adds a restriction and a liberation. The restriction is that we have to try to oppose what is established with every machine or device.

This attempt is sometimes very clear in the function of the device. For example, glasses designed to prevent facial recognition, inspired by the Ibarrola case; but on other occasions it is subtler and more conceptual: the act of “wasting” resources that industry needs and is very willing to pay for, in some other direction, or subverting the standard use of a device by drawing attention to euphemisms or hidden intentions.

In August 2019, Guillermo Ibarrola was identified by a Russian facial-recognition system contracted by the government of the City of Buenos Aires. He spent several days in prison until the mistake was noticed.

As for the liberation, Maker Counterculture frees us from the need to seek a consensual utility in devices. For example, it is easy to find consensus that adding a motor to a skateboard makes the resulting device useful, but consensus on the usefulness of a machine that makes rain will probably be harder to find, since it does not actually make it rain.

The inevitable question is: why make a machine that does not fulfill its function? To investigate, to collaborate, to draw attention to an issue – pluviculture, for example – to recover or revisit a history, as an electronic artwork of buried hyper-complexity; and ultimately, that slack between the machine’s name and its declared function can be full of meaning.

DÉTOURNEMENT

Why, then, does culture encourage technological studies when, increasingly, they contain the seed of disputing what is established? On the one hand, these technological studies have a high degree of specialization and specificity; on the other, those who receive them are often too indebted and artificially desperate to explore intersections between technological domains of knowledge or potential détournements without immediate reward.

Détournement refers to taking words, ideas, and objects and giving them a different use.

Perhaps one is a maker when they go with the flow of consensus and becomes part of the maker counterculture when their “making” explicitly intends to open a door to strangeness, disturb what is established, and enable improbable conversations.

NAVIGATING UNCERTAINTY

In the Maker Counterculture process, between starting ignorance and understanding there is often a liminal space where previous explanations are no longer enough and new ones have not yet stabilized. Moving through this

space requires managing states of confusion and frustration, developing tolerance for ambiguity without pausing or stopping the making. Building at the margins means working with incomplete documentation, unknown components, unpredictable results, and problems that may not have a solution.

At some point we begin to internalize a kind of Fuzzy Logic in which binary classifications are replaced by degrees of membership. Instead of moving from “not knowing” to “knowing,” we can understand a concept to some degree, apply it in certain contexts, and build even without yet understanding all of its foundations and possibilities.

Knowledge in Maker Counterculture often begins as partial and provisional, as an operational intuition, and through tests, errors, and observations, that intuition gradually gains precision.

A valid analogy is to dare to enter a territory with an incomplete map, trusting that we will find clues, help from the community, and eventually develop our own mechanisms for finding our way.

ACTIVISM

As activism, Maker Counterculture is a path of resistance that allows us to reject the credibility of that appearance in which there is no alternative but to meekly submit to the brutality of technology. It is using technology to fight

technology itself when it oppresses humanity and creative possibilities, and when it reduces alternatives to a series of equally frustrating options.

MEETUPS

From the perspective of control, the confinement experiment was successful. More and more people work from home and remain there, feeling that going out is inconvenient, expensive, and risky. The obvious point is that when communication with others is limited to networks, we are more vulnerable. Meeting people, discovering the person behind their “persona” on social networks, and sharing a physical space has a special benefit: we discard a portion of the imposture and the simulable.

Although meeting does not need any excuse beyond a shared obsession or a fun activity, here are some proven and engaging formats within Maker Counterculture:

Repair meetups

Talk about obsolescence, recover and extend the useful life of electronic devices. Learn to diagnose, solder, and repurpose device parts.

Privacy meetups

TOR, PGP, encryption, password managers, firewalls, secure communications, and reducing digital footprints

Maker meetups without Internet or AI

What devices are we capable of developing without Internet access or using LLMs?

DIY music meetups

Circuit bending, MIDI, synthesis, effects

Obsolete technology meetups

Explain, test, and integrate supposedly obsolete electronics

Technology sovereignty meetups

What technologies were proposed in peripheral regions? What made them succeed or fail? What could be proposed today while considering the needs and particularities of a region?

INSTRUMENTAL REASON

Instrumental reason is a concept developed by the Frankfurt School and is defined as:

The efficiency of means for achieving a predetermined objective, without questioning the rationality or value of the end in itself

It is something deeply internalized in modernity and can be visualized as the shortest path between a problem and its solution.

The cult of economic efficiency and functionality leads us to repeat the most traveled paths, prevents creativity, and reduces us to means for the productive apparatus.

For Maker Counterculture, more important than mastering specific technologies is freeing ourselves from instrumental reason.

That is, emancipating ourselves from the “default self” as a cog in a machine so large that nobody knows what it is for. This liberation is a subtle but fundamental difference between playing with an Arduino board that will ultimately end up at the bottom of a drawer and making, as a discovery, an identity and an act of resistance.

ELECTRONIC ART

Although there are Maker Counterculture devices indistinguishable from electronic art, and electronic art that could easily be Maker Counterculture, the difference does not lie in the aesthetics or the technology used, but in the conditions of origin and validation.

Generally speaking, Maker Counterculture devices are not conceived for exhibition. They do not follow the logic of installation, institutional circulation, or curatorial interpretation. They usually emerge as personal solutions, technical experiments, proof-of-concepts, or even obsessive artifacts, where function, exploration, or technical challenge precedes any exhibition concern. This translates into a recurring characteristic: degrees of complexity that are excessive or out of proportion to the visible result. There is more code than necessary, more electronics than the final effect would justify, more engineering than narrative.

In electronic art, even in its most experimental forms, complexity tends to be domesticated by the viewer's experience.

In Maker Counterculture, by contrast, complexity is usually directed inward: it satisfies the maker, documents a process, explores a technical limit, or solves a problem without consensus.

MAKER TEMPLATES

A maker is someone who makes, not someone stuck in an endless loop thinking about what to make. Yet creativity requires conditions contrary to this modernity. There must be slack in routines, courage to embark on paths that may not prove productive, emancipation from instrumental reason, and patience, since creativity is a practice. While personal conditions are being modified to enable creativity, Maker Counterculture proposes a series of templates for identifying ideas and getting to work.

The Hunter Method

Hunter S. Thompson was a writer and journalist, creator of Gonzo journalism. When he wanted the feeling of writing a great novel, he took *The Great Gatsby* and typed it letter by letter and word by word. In the same way, we will take a well-documented project and replicate it with the intention of appropriating it at some point by modifying components or functional aspects; and if that does not happen, we will still have gained skill and experience.

Crossover

It means taking something that exists digitally and bringing it into the

physical world, or taking something mentioned in fiction and making it real. In the latter case, fictional principles may carry over, but as long as this is not concealed, there is no problem in moving forward with the device. One example of this template is *La máquina de Klausner*. Inspired by Roald Dahl's story *The Sound Machine*, this device reads soil moisture conditions and maps the measurements to audio.

Sometimes we seem to forget that science has more to do with discoveries than with facts, and fiction is often capable of anticipating those discoveries or pointing toward where to look. Roald Dahl's story and the work of Jagadish Chandra Bose inspired a CIA interrogator named Cleve Backster, who applied a polygraph to plants in 1968. His methods were challenged by science. However, professors Yossi Yovel and Lilach Hadany of Tel Aviv University were able, with the help of Machine Learning, to demonstrate that plants do in fact emit messages when they are stressed.

Upcycling

It is a variation of recycling in which a device is taken and used to build an improved device. For example, a soldering iron tip and the slow-rotation motor of an electric oven are used to build a desktop micro-shawarma.

Extravagance

A modest problem is solved with an extravagance of resources. For example:

the need to keep the access step to a home clear results in a client-server system with Machine Learning-based recognition, Bluetooth communication, and progressive alerts.

Autonomy with AI

Given a particular problem, an autonomous machine with AI is developed. An example of this template is the Reggaeton Be Gone device, which identifies a certain musical style with Machine Learning and attempts to disconnect the source or degrade the sound quality.

VS

In our digital existence we often believe that we found something or reached it by chance when in reality it was placed there to intercept us. We are ultimately a sum of patterns, and platforms, with the help of Machine Learning, exploit those patterns to steer us in one direction or another, limiting our natural paths, our processes, and our creative possibilities. VS (Volquete Surprise) is an activity that emerged at Maker Counterculture gatherings to recover the random element of discovery.

The procedure is simple: take random paths through unusual areas, paying attention to discarded technology beside dumpsters. An old printer, a Walkman, or a container full of camping lamps can become raw material,

inspiration, or the rabbit hole toward the next project.

ASKING FOR HELP

Although Maker Counterculture is collaborative – we write tutorials, share code, and help others – there are ways to achieve better results from this help. You learn this over time, but it is better to save yourself the trouble.

Before contacting the author of a project, gather information: which board, which software version, and what the specific error is, for example.

It is also useful to search for the error and possible solutions. The author may already have answered it several times, or a similar problem that has already been solved may save them more work.

Contacting privately is always the last resort. If the author uploaded the project to a platform with public messaging, write there.

In the message, in addition to gathering information, try a hypothesis, mention an unsuccessful test, something that reflects an effort to solve the problem.

Finally, never forget to acknowledge the work already done. After all, the author will probably help, but they are not obligated to do so.

What is the point of consulting a maker when generative AI is available? Leaving aside the fact that some “makers” use generative AI wholesale without understanding anything that was published – and AI may be the place to turn for those cases – consulting the maker produces feedback, clarifications, and moves the project in some humanly positive direction. On the other hand, there are contents that are too marginal for generative AI to provide a satisfactory answer in any case.

ARTIFICIAL INTELLIGENCE

"People fear that computers will become too intelligent and take over the world, but the real problem is that they are too stupid and have already taken over the world"

Pedro Domingos

We call ourselves *homo sapiens* (wise human), so intelligence must occupy an important place in our species, and something called Artificial Intelligence puts our assumptions, roles, and relationships into dispute.

But what are people talking about when they talk about AI? Even for those who work with technology, it is sometimes not very clear. To define AI, it might be useful to define intelligence, which is not so simple.

Intelligence

What does it mean to be intelligent? An intuitive approach may relate it to the ability to think. Yet organisms such as slime mold can find their way out of a maze without a central nervous system, that is, without a brain. Let us say, then, that intelligence is “doing the right thing at the right time.”

Artificial intelligence

Now what is “artificial intelligence”? There are many perspectives and definitions. For our purposes, let us say that artificial intelligence is an artificial system that makes intelligent decisions based on some type of input. Example: braking an autonomous vehicle when the distance to an obstacle is less than X centimeters.

In this view of intelligence there is usually a precondition (if the distance is less than X) and an action (brake).

Is AI intelligence?

“Asking whether machines can think is as relevant as asking whether submarines can swim”
Edsger Dijkstra, 1984

In 1770 Von Kémpelen invented the Mechanical Turk, an intelligent automaton for playing chess that won games all over Europe. It was later discovered to be a fraud: a small person was inside playing with an ingenious system of pulleys and mirrors. Many years after the Mechanical Turk, IBM was able to create a real machine called Deep Blue, capable of beating any chess champion. At that point critics began to say that winning at chess was not representative of intelligence. This phenomenon is called the “AI Effect”.

chess with which it won games all over Europe. It was later discovered to be a fraud:

Whenever Artificial Intelligence researchers find a way for a computer to perform a task, critics say that task is not representative of intelligence.

Grasping the concept of intelligence is counterproductive because

humans keep moving the goalposts whenever it threatens us. In any case, from the technological point of view the interest is not in “creating machines that think like humans” but in enabling computers to carry out tasks that used to require human intelligence.

Grasping the concept of intelligence is counterproductive because humans keep moving the goalposts whenever they feel threatened. In any case, from the technological point of view the interest is not in “creating machines that think like humans” but in enabling computers to carry out tasks that used to require human intelligence.

Weak and strong AI

In philosophy there is a division of AI into weak and strong. Weak means that machines can act as if they were intelligent. Strong AI means that machines are actually intelligent and thinking.

Machine Learning

In terms of applications, AI can be considered an umbrella containing many things, one of them – and not the best understood – being Machine Learning.

Machine Learning is a way of discovering patterns about how the world works. Machine Learning learns from historical data without needing to

be explicitly programmed.

We use Machine Learning, for example, when we teach a device to recognize faces in a photograph, sounds, or combinations of gases. Why not use traditional heuristic programming? In a hypothetical case, we need to detect whether a room is tidy or messy. We take photos of the messy room in many different ways. Then we take photos of the tidy room in many different ways. Once the photos have been taken and classified, analyzing the differences will require an image expert and a great many rules. And after all the time and money invested, the system will not work well. The same thing is very easy to solve with Machine Learning. We take many photos of the tidy room and many of the messy room. From this point, Machine Learning finds the patterns automatically. We can even test whether the training was successful by showing it photos it has not seen before and determining the percentage of correct and incorrect detections.

Data in Machine Learning

In Pedro Domingos's book *The Master Algorithm*, the idea is expressed that computers enabled the Internet, the Internet enabled large-scale data capture, and large-scale data capture enabled Machine Learning. Data is therefore the building block of Machine Learning, and this gives it a special responsibility.

Compiling a dataset is typically the most complicated, expensive, and time-

consuming part of any project. It is also where difficult-to-detect errors can occur that contaminate the entire project.

Datasets must have certain characteristics:

Relevant: datasets must contain information useful for the problem to be solved. Example: in a system that uses heart rate to estimate athletic performance, the dataset must contain heart-sensor data, not soil-moisture sensor data.

Representative: datasets must contain information about all conditions. For example, at a passport-renewal kiosk in Australia, the possibility of citizens of East Asian ancestry was not considered, preventing them from taking their photos because the kiosk detected their eyes as closed.

Balanced: although Data Augmentation techniques exist, in general, if data are too unbalanced the results will not be satisfactory. This happened, for example, with Google Photos and the controversy over the “gorillas” label.

Reliable: a dataset must be consistent. If it contains errors, they should be uniform. If there is noise, it should be of the same type and magnitude found in the real world. For example, if a project runs out of sensors and half are ordered from a different manufacturer with lower quality, that could introduce noise that contaminates the project.

Overfitting

Overfitting is the effect of over-training a learning algorithm to the point where it finds a pattern that is not true in the real world.

Generative AI

Although Machine Learning has useful and solid practical applications, it often goes unnoticed except by specialists. Generative Artificial Intelligence, on the other hand, receives all the attention. Anyone can write a prompt and “magically” obtain text, music, code, and images.

Generative AI is therefore a branch of AI – sometimes mistakenly thought to be all of AI – that focuses on generating original content from existing data. GenAI models learn patterns and structures in training data and generate new data with similar characteristics.

The results of generative AI are often so surprising that they can lead to confusion about their nature. Such confusion encourages people to assign characteristics to it that it simply does not possess. One way to resist this effect is to think of generative AI as a sophisticated predictive system. Just as a search engine text box shows the next most likely word.

LLM

At the center of GenAI are LLMs. An LLM is an advanced artificial intelligence system specialized in processing, understanding, and generating human-like text. These systems are typically implemented as a neural network trained on enormous amounts of text, allowing them to learn complex language patterns and perform tasks such as machine translation, creative text generation, answering questions, summarizing text, and other tasks related to reasoning and language.

These spectacular tasks are carried out through predictions. For example, given the prefix “The most famous city in the United States is...”, the LLM will probably assign a high probability to “New York” and “Los Angeles” and a low probability to “apple” and “laptop”.

It is possible to create a basic LM using an Ngram table. Ngram tables are based on Claude Shannon’s probabilistic models for analyzing language. An Ngram model predicts X_i based on X_{i-1} , X_{i-2} , etc., usually assuming that X_i depends only on the last n values in the sequence in order to save computational resources. When $n=1$, the Ngram is called a Unigram and is as simple as the following table:

```

“Today” 0.2
“Microphone” 0.05
etc

```

Transformers

A more modern LM will be based on transformers: a type of neural network that can process token sequences in parallel thanks to a “self-attention” mechanism. This technology was developed by Google in 2017 for text translation and is a sequence-to-sequence model. In other words, it converts sequences from one domain into another. For example, a sentence from English to French. The original transformer architecture consists of two parts: an encoder and a decoder. The encoder converts the input text – the English sentence – into a representation that is then sent to the decoder. The decoder uses that representation to generate the output text autoregressively.

Pre-training

Pre-training is the foundational stage in which an LLM is trained on massive, diverse, unlabeled datasets. The task is to predict the next token given a previous context. This stage is slow and expensive. After pre-training, fine-tuning can be performed more quickly and economically. Specific datasets with positive examples for certain tasks are used.

Reinforcement

After fine-tuning, a second layer of the same process can be performed, called reinforcement learning from human feedback, or RLHF. This is where, for example, safety filters can be incorporated to prevent the LLM from responding in politically incorrect ways. Unlike the previous step, negative examples can be used here.

Looking more deeply, it becomes clear that LLMs are sophisticated predictive systems. So where does the attribution of extra qualities come from? On the one hand, LLMs often display “emergent behaviors,” meaning the ability to solve problems for which they were not specifically trained.

On the other hand, there is something called the Eliza effect.

The Eliza effect can be defined as the tendency to unconsciously assume that computer behaviors are analogous to human behaviors.

The Eliza effect emerged in 1966 from the conversational bot ELIZA, developed by MIT computer scientist Joseph Weizenbaum.

Context and vector search

A perhaps induced confusion is to assume that an LLM learns from our interactions and comes to “know” us. In reality, a conversation does not

modify the model's parameters. During a session, the system can use previous content as context and, depending on the application, may also have some external memory mechanism that retrieves relevant information about earlier interactions. That information is simply added to the context before generating a new response.

One possible analogy is that we are introduced to a person, talk all day, and the next day that person does not remember who we are or what we discussed, unless they have some kind of external memory: a notebook, a database, or some other record they can consult.

Although a traditional search from the pre-LLM era could use techniques such as inverted indexes, stemming, and TF-IDF to find relevant documents, it still depended on the relationship between query terms and terms in documents. For example, a search for “automobile” might have trouble retrieving an email that mentioned the word “car”.

Vector search introduces another approach. First, an embedding model transforms texts – words, phrases, documents, or complete queries – into numerical vectors within a high-dimensional space. Thus texts representing related concepts tend to be close to each other in that space. Given a query, the system generates its vector and compares it with stored vectors using some similarity or distance measure, such as cosine similarity or dot product.

Thus the system transforms the problem from “does this word appear?” into something more like “how close is this content to the concept I am looking for?”

An email that talks about “buying a new vehicle” will be shown as relevant to a query about “automobiles”, even though neither expression matches exactly.

This mechanism is one of the fundamental components of RAGs (Retrieval-Augmented Generation): first, relevant information is retrieved through search — generally vector search — and then that information is incorporated into the context received by the LLM.

With this trick, language models reinforce the idea that they are not only capable of locating content through a keyword, but also understand concepts or similarities that are less obvious.

When did AI begin?

There is an established version of the story in which AI is a revolution dating to around 2022 with the appearance of ChatGPT, but it is possible to go back and find:

2017: transformer model paper

1997: Deep Blue defeats Kasparov

1986: the backpropagation algorithm is developed

1958: the Perceptron is invented

1956: the term AI is coined at the Dartmouth conference

1911: Frederick Winslow Taylor publishes Principles of Scientific Management

AI risks

AI is often presented as the solution to every problem, even though it also adds its own problems, sometimes of a subtler nature but no less significant. AI is also a kind of battlefield, and in every battle there are as many deceptions and stratagems as winners and losers.

Nominal risk: Artificial Intelligence is called that, but it could just as well have been called something else: a classification-inference algorithm or stable-diffusion predictive generation algorithm in some of its branches, for example. For some strange reason, people tend to take Artificial Intelligence too literally. As if merely carrying the word intelligence meant that every output of an AI system was necessarily intelligent. A Machine Learning system trained on imbalanced data, for example, could produce an absurd output.

Data risk: in 2015 Google introduced a novel feature in Google Photos: AI-

based automatic labeling. Some African American children found that they had been labeled “gorillas”, which caught their attention because they had not been to a zoo and it turned out the photos were of them. Apparently, certain categories were underrepresented.

Human risks: in 2019, a Russian facial-recognition system identified Guillermo Ibarrola as a fugitive. Ibarrola insisted he had nothing to do with the incident, but he trusted what the AI said and was jailed. It was later discovered that an employee responsible for entering the data had entered the suspect’s middle name incorrectly.

Explainability risk: in Machine Learning, pattern detection is performed automatically. This creates a “black box” character that makes it impossible to explain why a particular decision was made.

Sim2real risk: just as there is data, there is synthetic data. Synthetic data is not obtained naturally, but simulated or even generated with generative AI. This synthetic data, often indistinguishable from real data, is used to train or test models with unpredictable loops and effects.

ELECTRONICS

Electronics is fascinating and the foundation of computers. Understanding its fundamentals provides greater depth in understanding technology as well as in building devices.

Conductor and insulator

A material with high resistance to electricity is known as an insulator. Most plastics are insulators. A material with low resistance is a conductor. Metals such as copper, aluminum, silver, and gold are good conductors.

Measurements

Electrical pressure is measured in volts. Electrical flow is measured in amperes. Resistance is measured in ohms.

Ohm's Law

volts = ohms $\times$ amps

$$V = I \times R$$

$$I = V/R$$

$$R = V/I$$

V is the voltage difference between two points in a simple circuit. I is the current in amperes flowing through the circuit between two points. I is used because “inductance” was previously used. R is the resistance in ohms between two points.

Ohm’s Law is useful, for example, to determine whether a component is going to burn out.

Watts

A watt is a unit of work. Engineers have their own definition of work: they say work is performed when a person, animal, or machine pushes something to overcome mechanical resistance.

When electrons make their way through a circuit, they overcome a kind of resistance and therefore perform work that can be measured in watts. Lamps and amplification, for example, are measured in watts.

$$\text{watts} = \text{volts} \times \text{amps}$$

Using the standard symbols

$$W = V \times I$$

$$V = W/I$$

$$I = W/V$$

Series and parallel

When resistors are connected in parallel, the resistance is added. When they are connected in parallel, the resistance is divided. When two batteries are connected in series, the voltage doubles. When two batteries are connected in parallel, they provide the same voltage for twice as long.

Direct and alternating current

The current from a battery is called direct current (DC). It is a constant flow in a single direction, similar to water flowing from a faucet. The current from a household outlet, on the other hand, is completely different: it changes from positive to negative 50 times per second in Argentina (in some countries, 60 times per second). This is known as alternating current (AC).

Household outlets usually have 3 pins. A is Live: it provides the voltage that alternates between positive and negative relative to pin B, called Neutral. If a device has a fault, it should be protected by shunting the voltage to pin C: Ground.

Alternating current is fundamental in certain cases, such as distributing electricity over long distances. It is also useful in motors and household appliances. Most simple electronic circuits operate on direct current because it is easier to handle.

Switch

As its name indicates, switches interrupt or connect terminals. They are commonly used, for example, to turn on a device (by connecting the power supply). They can have 2 or 3 terminals. In the case of 3 terminals, they allow each end to be connected to the middle terminal.

Resistor

As its name indicates, a resistor “resists” the flow of electricity. Resistors have a color band code that is read as follows. First, place the silver or gold band on the right. That band indicates tolerance: silver 10%, gold 5%.

Electrical resistance is measured in ohms. The Greek letter omega (Ω) is used to indicate ohms. The letter K (or $K\Omega$) means one kilohm, which is 1,000 ohms, and the letter M ($M\Omega$) means one megohm, which is 1,000,000 ohms.

Ohm is used because Georg Simon Ohm was able to define the fundamental relationship between electrical voltage, current, and resistance in 1827. This is known as Ohm's Law.

The first two bands use this coding

Black 0

Brown 1

Red 2

Orange 3

Yellow 4

Green 5

Blue 6

Violet 7

Gray 8

White 9

The third band tells how many zeros to add

Black - No zeros

Brown 0 1 zero

Red 00 2 zeros

Orange 000 3 zeros

Yellow 0000 4 zeros

Green 00000 5 zeros

Blue 000000 6 zeros

Violet 0000000 7 zeros

Gray 00000000 8 zeros

White 000000000 zeros

A brown-red-green resistor would have values 1-2 and five zeros, resulting in 1,200,000 ohms, or 1.2MΩ. An orange-orange-orange resistor would have values 3-3 and three zeros, resulting in 33,000 ohms, or 33KΩ.

Diode and LED

A diode is a component that allows electric current to flow in only one direction. An LED is a light-emitting diode. LEDs are inexpensive, long-lasting, available in different colors, and commonly used in electronic projects to indicate information: power-on status, for example. As a mnemonic, the longer LED lead should receive more positive voltage than the shorter lead. The voltage difference between the leads and the current must not exceed the manufacturer's specification.

Potentiometer

Potentiometers are components that allow resistance to be varied (and therefore voltage and current). They are commonly used, for example, to

adjust the volume of an audio device. They usually have three contacts: the outer terminals form the voltage divider and the middle contact is the output with the applied resistance.

Capacitor

A capacitor is like a small rechargeable battery. Because of its size, it charges in a fraction of a second. Capacitance (electrical storage capacity) is measured in farads. DC current does not flow through a capacitor; instead, voltage accumulates very quickly and remains when the source is disconnected. In electronics, ceramic capacitors are commonly used (discs, non-polarized) and electrolytic capacitors (cylinders, polarized).

Relay

The concept of switching is fundamental in electronics, and switching should be understood as using one flow of electricity to control another. Digital devices could not exist without this principle. A relay is a typical switching case in which a smaller current is used to control a larger one. For example: a 5V DC signal is sent to enable the flow of 220V AC. Inside a relay, a coil commonly generates magnetic attraction and closes two contacts.

Transistor

This component was invented at Bell Labs in 1948. It can interrupt and enable the flow of electricity like a relay, but it is more sensitive and versatile. A transistor is a semiconductor in the sense that it conducts electricity under certain conditions. Transistors have three legs: emitter, base, and collector. The collector receives current, the base controls it, and the emitter sends it. Internal resistance varies according to the power applied to the base. NPN transistors turn on with positive voltage and PNP transistors with negative voltage at the base. Bipolar transistors amplify current, not voltage. A small fluctuation at the base enables a large change in current between emitter and collector. One limitation of the transistor is that it is off until turned on, whereas a relay can remain on until it is turned off.

The 2N2222 transistor is the most widely used semiconductor in history. It was developed by Motorola in 1962 and is still manufactured. Since the patent has expired, any company can manufacture it.

Ampere-hours

Batteries usually indicate Ah, which means ampere-hours. A 9V battery is rated at 0.3 amp-hours while delivering 100 mA. An AA battery is rated at 2.2 amp-hours while delivering the same 100 mA.

Adding a resistor to a capacitor makes the capacitor take longer to charge and consequently provides timing. In this way, electronic components are combined to form complex circuits.

ARDUINO

Arduino is an open-source electronics platform developed in Italy. Arduino boards and derivative boards usually have a series of power pins (3.3V, 5V, VIN, GND), communication pins (SDA, SCL, UART tx, UART rx, etc.), and generic input/output pins, both digital and analog (D2, D3, A0, etc.). Digital pins can only receive and transmit 0 and 1, while analog pins can have a range from 0 to 5V with a mapping from 0 to 1023 through the ADC. Through these pins it is possible, for example, to connect a switch or a humidity sensor, process those readings, and generate some kind of output such as turning on a light, moving a motor, or publishing something on the Internet. To tell the Arduino what to do, a series of instructions in the C++ programming language are uploaded to the board's microcontroller. Once powered, it automatically begins executing that code.

PARTS OF AN ARDUINO PROGRAM

The C++ code for an Arduino consists of two parts: setup and loop. The first part is executed only once and, as its name indicates, contains the “setup” or initial configuration. The second part is executed repeatedly, or in a “loop”. This is important for deciding where to place the different parts of our code.

If what we want to program only executes once, it can be entirely inside setup and the loop function can remain empty, or it can be placed inside loop with

some mechanism (conditional or subloop) so that it is executed only once.

VARIABLES AND CONSTANTS

```
int ledVerde=2;  
const float pi = 3.14;
```

The first line means that ledVerde is of type Integer and is assigned the value 2. The second means that pi is a float constant with the value 3.14.

All lines in Arduino code must end with a semicolon.

PINS

```
pinMode(2, OUTPUT);
```

With pinMode, the pin number is indicated first and then whether the referenced pin is an input or an output. The pin number can be specified

directly in the function – `pinMode(2,OUTPUT);` – but it is better to define it in a variable to make modifications easier. This definition is used, for example, to connect an LED.

If, for example, we want to read a switch, the pin definition will indicate `INPUT` or `INPUT_PULLUP`, which adds an approximately 20k ohm resistor between the PIN and 5V.

PWM

Since Arduino cannot vary the output voltage of its pins, it is necessary to use a technique called Pulse Width Modulation, or PWM, to, for example, smoothly fade an LED's light. PWM makes an output pin rapidly vary its voltage from high

to low (between 5 and 0V) over a fixed period of time. This voltage change happens so quickly that the human eye cannot see it. When the pin voltage rapidly changes from high (HIGH) to low (LOW) over a fixed period, it is as if we could vary the voltage level of that pin. The percentage of time a pin is HIGH relative to the total time (period) is known as the duty cycle. When the pin is HIGH for half the period and LOW for the other half, the duty cycle is 50%. A low duty cycle makes the LED much dimmer than a higher duty cycle. The pins that can be used for PWM are indicated with the ~ prefix.

```
analogWrite(9,127);
```

This PWM instruction indicates the output pin first and then the duty cycle (from 0 to 255).

COMMENTS

Using two leading slashes defines comments in the code, which is ideal for documenting what is being done. For example:

```
// pin initialization
```

SERIAL MONITOR

The Arduino may not be connected to any output device such as displays or actuators and we may still need to read a variable related to execution. For this purpose we can use the Serial Monitor.

```
Serial.begin(115200);  
Serial.println("Hello world");
```

The first line indicates that a serial communication line is opened at 115200 baud. The second line sends the text “Hello world” from the Arduino to the Arduino IDE (software installed on PC, Mac, or Linux).

CONDITIONALS

Conditionals are sections of code that execute only when a condition is met. It is important to note that equality comparison uses two equals signs, to distinguish it from assignment. The condition in the example is: when the Relay state is HIGH, change it to LOW and vice versa.

```
if (relayState==HIGH){  
    digitalWrite(relay, LOW);  
}  
else{  
    digitalWrite(relay, HIGH);  
}
```

LOOPS

Regardless of the fact that the main Arduino code runs in Loop over and over again, it is often necessary to define internal loops, and this is done with the while instruction and a condition specified in parentheses.

```
while(relayState==HIGH){  
  // code to repeat goes here  
}
```

FOR

When it is necessary to repeat a segment of code while iterating through counter values, the following code can be used.

```
for(int i=1; i<=10; i++){  
  // code that runs from 1 to 10  
}
```

FUNCTIONS

Functions are ideal for reusing parts of code. In the Arduino IDE they are defined by prefixing with `function`, and the function's return value is produced with the keyword `return`. The following example shows a function that replaces the degree symbol with the word "degrees".

```
function removeSpecial($str){  
  $str=str_replace("°"," degrees ",$str);  
  return $str;  
}
```

ESP32

The ESP32 was created in 2016 by a Chinese company that had already gained popularity with the ESP8266, called Espressif Systems. Unlike Arduino, which was originally developed as an educational platform, the ESP32 was designed as an SoC (System on Chip), integrating Wi-Fi and Bluetooth on the same chip along with a 32-bit processor, more memory, and peripherals. At the time, while a traditional Arduino Uno needed additional hardware to connect to a network, the ESP8266 and ESP32 could do so directly. The popularity of the ESP32 did not come from an Espressif marketing strategy, but from the Maker community: low cost, Arduino IDE support, and clones

made the ESP32 one of the most widely used open-hardware platforms.

It is important to note that there are also Arduinos with WiFi, such as UNO R4 WiFi, UNO Q, and even an Arduino Nano ESP32.

UART

UART – acronym for Universal Asynchronous Receiver/Transmitter – is one of the most widely used communication protocols between devices. It can be found, for example, in connections between microprocessors and GPS or Bluetooth modules, as well as in various hardware devices as a diagnostic mechanism or console access point. UART is a communication protocol based on asynchronous serial transmission with configurable speed. Asynchronous means that there is no shared clock signal between transmitter and receiver. The transmitter generates the bit stream using its own clock and the receiver uses its own to receive the data. Synchronization is maintained by configuring the same baud rate on both devices.

SERIAL AND PARALLEL

In a parallel digital transfer, all bits travel simultaneously from source to destination because multiple lines exist between transmitter and receiver. In serial communication such as UART, by contrast, data is sent bit by bit using a single line. And because communication is usually bidirectional, two wires are used. The TX of the first device must be connected to the RX of the second, and vice versa. The transmitting UART receives parallel data from the CPU (microprocessor or microcontroller) and converts it to serial. That serial frame travels to the receiving UART, which converts it back into parallel data to deliver to the CPU. UART organizes transmission into packets. Each packet includes a start bit, a data frame, a parity bit, and one or more stop bits.

AT COMMANDS

AT commands are a simple text language created in 1981 for Hayes modems.

The famous letters AT stand for Attention.

Example:

```
AT (Are you there?)  
OK (Yes)
```

The idea behind AT commands was simple and elegant: the modem is always in one of two modes. Data mode (transparent: everything that arrives is sent to the telephone line). Command mode (allows configurations to be changed, etc.). AT commands are usually sent through UART serial connections and can be found in modems, Bluetooth modules, drones, and even Starlink satellites or ESP32 modules.

Examples of AT commands for ESP32 modules:

```
Scan networks AT+CWLAP  
Send data TCPAT+CIPSEND=tamaño
```

AT's simplicity, readability, and compatibility gave it the status of "Esperanto" of embedded communication devices.

UART HACK

Regardless of its standard use for communicating sensors, for example, which is usually well documented, UART can be found in various devices as a diagnostic access point and an entry point for loading modified firmware. Sometimes PCB boards have the UART connection labeled directly as such or with TX, RX. At other times it can be inferred using a multimeter. Set the multimeter to continuity mode and look for GND by probing some part of the PCB that is clearly GND. Then distinguish TX and RX by placing the multimeter in DCV mode, one probe on GND and the other on the pin to identify. If it is RX it will remain high and close to VCC unless we are sending data to it. If it is TX it should fluctuate. Regardless of the pin locations, the baud rate, parity bit, and flow control then need to be tested. With the inferred settings, a USB-to-TTL adapter and a terminal can establish UART communication with the device.

LORA

LoRa was created in France in 2009 to enable low-speed, long-distance communications using high-frequency radio waves with minimal power consumption. It is a physical modulation layer that, together with the LoRaWAN network protocol, creates a complete communication architecture for IoT.

Many maker technologies come from countries other than the United States or China. LoRa was developed in France. Arduino in Italy, Raspberry Pi in England, and Prusa in the Czech Republic.

CHIRP

LoRa does not use traditional frequency-shift keying (FSK) modulation, but rather a proprietary modulation known as CSS (Chirp Spread Spectrum), in which the data signal is spread over a much wider bandwidth than is strictly necessary for the raw information. This makes the signal highly robust against noise and interference. LoRa operates in unlicensed ISM (Industrial, Scientific and Medical) radio-frequency bands, which vary by region and are subject to local regulations:

Europe 863-870 MHz

North America 902-928 MHz

Asia/Australia 915/928 and 920/923 MHz

Within these bands, multiple channels are defined. For example, in Europe, base channels commonly used are 868.1 MHz, 868.3 MHz, and 868.5 MHz.

LoRa modules commonly communicate with Arduino and Raspberry through AT commands via UART. For example: AT+SEND=7,10,HOLA MUNDO means send 10 characters, Hola Mundo, to address 7.

MESHTASTIC

Initiatives such as Meshtastic use LoRa to build off-grid communication networks without depending on the Internet or cellular infrastructure. Each device acts as a network node and can relay messages from other nodes, forming a mesh that can cover much greater distances than a direct connection could reach. A message may, for example, travel from one device to another, be relayed by several intermediate nodes, and finally reach its recipient.

Meshtastic incorporates encryption mechanisms to protect message content, where intermediate nodes can participate in routing without needing to know the contents of encrypted communication. This enables an interesting architecture from the standpoint of technological autonomy: we can build a messaging network that continues to work even if the Internet and cellular service go down.

It is important to note that even if content is end-to-end encrypted, the existence of transmissions themselves, their radio characteristics, and certain information required for the network to operate can reveal metadata.

BINARY SYSTEM

“Od glifulwkdg hv sursruflrqdo do sholjur”

Mhdq Edxgurloodug

The difficulty is proportional to the danger

(Jean Baudrillard)

A binary system is essentially a number system in which numbers are represented using 0 and 1. This simplicity makes it ideal for working with computers, which are themselves based on electronics using switches to store and manipulate files. Despite the simplicity of the binary system, any kind of data can be represented with 0 and 1: text, images, and sound.

0 is 0

1 is 1

2 is 10

4 is 100

5 is 101

BINARY TO DECIMAL

1 0 1

4 2 1

We add the values that have 1: $4+1 = 5$ (2 is not added because its value is 0)

Another example

1010010

1 0 1 0 0 1 0

64 32 16 8 4 2 1

We add the values that have 1: $64 + 16 + 2 = 82$

DECIMAL TO BINARY

Divide by 2 and, if there is a remainder, write 1. Otherwise write 0. Then reconstruct it in reverse order.

Example for the number 5

$5 / 2 = 2$ Remainder: 1

$2 / 2 = 1$ Remainder: 0

$1 / 2 = 0$ Remainder: 1

Then reconstruct in reverse: 101

Example for 6

$6 / 2 = 3$ Remainder 0

$3 / 2 = 1$ Remainder 1

$1/2$ 0 Remainder 1

Reconstruct in reverse: 110

Example with 11

$11 / 2 = 5$ Remainder: 1

$5 / 2 = 2$ Remainder: 1

$2 / 2 = 1$ Remainder: 0

$1 / 2 = 0$ Remainder: 1

Reconstruct: 1011

LINUX

The Linux command line is a text interface to the computer. Shell, terminal, and console are some synonyms for the command line. Learning the Linux command line is important, on the one hand, for building devices with Linux. But beyond that specific application, the command line has existed since the 1970s, uses minimal resources, and still works more or less in the same form. Graphical environments, on the other hand, change at a pace with little continuity or connection.

Knowing the command line lets you reuse any computer capable of running Linux and automate complex tasks by combining different commands in a shell script.

Any computer running Linux has a command line. The way to access it may vary, but in general you can click an icon in the graphical environment or press CTRL-ALT-T. In the case of a Raspberry Pi without a graphical environment, the board boots directly to the command line.

Any computer running Linux has a command line. The way to access it may vary, but in general you can click an icon in the graphical environment or press CTRL-ALT-T. In the case of a Raspberry Pi without a graphical environment, the board boots directly to the command line.

pwd

It is short for Print working directory. It gives us the current path.

With Change Directory, we move to the root directory / and then to home.

With Change Directory, we move to the root directory / and then to home.

cd ~/Desktop

With this prefix, which means home, we move from home to Desktop.

mkdir /home/Desktop/test

With the Make Directory command, we create the test folder.

ls

With this command (list), we list the contents of the current directory.

ls > listado.txt

With this command we can redirect the output of list to a file. Redirection is a very powerful function commonly used in shell scripts.

cat listado.txt

With this command we can view the contents of the file listado.txt. Cat comes from conCATenate.

cat test1.txt test2.txt

This command concatenates test1 and test2 and displays them on screen.

echo "Hello world"

This command prints whatever is between quotation marks.

Wildcards

("?") is used to indicate any character within a filename. ("*") is used to indicate 0 or more characters.

cat test?.txt

This command will therefore print test1.txt and test2.txt.

Up arrow, down arrow, and TAB

Pressing the arrows makes the command line cycle through previously executed commands. Pressing TAB can complete information from files in the current directory. Example: if we type cat te and press TAB, it will complete cat test.

less text0Largo.txt

With this command it is possible to page through a long text. It is called less because there used to be one called More that only allowed moving forward through the pages.

Case

Linux is case sensitive. This means that UNO.TXT and uno.txt are two different files.

mv test.txt foldera

This command moves test.txt to foldera.

cp file.txt backup.txt

This command copies a file under another name to the same location or to another location.

rm test.txt

This command deletes (removes) test.txt. It can also be used for directories.

wc -l test.txt

This word count command allows you to count words. With the -l option it counts lines.

ls ~ | wc -l

This command allows the output of one command, such as a file listing, to be sent as input to another command (counting the number of lines).

man uniq

This command lets you consult the manual. In this case, the manual for uniq.

sort test.txt | less

This command sorts the contents of test.txt and pages the result.

sudo cat /etc/shadow

This command runs cat with superuser privileges. Sudo is commonly used to install new software, for example `sudo apt-get XXXX`. Why use sudo instead

of the root user? To avoid errors that could trigger problems for the OS.

Hidden files

To hide a file, simply give it an initial `.` This is used, for example, to keep system files hidden that could interfere with user files in certain directories. With `ls -a` it is possible to list all files, including hidden ones.

PROGRAMMING



For this appendix, the Python programming language was selected. Python is an interpreted language, meaning programs can be written with a text editor and then interpreted for execution. It is easy to learn, flexible, cross-platform, and has a huge number of packages to extend its capabilities. There are packages for scientific computing, graphics, and machine learning, for example.

What is the point of learning to program in Python or another language when an LLM can solve practically any assignment? On the one hand, learning to program is formative: it develops our abilities and our understanding of the world we inhabit. On the other hand, even when we obviously use LLMs for programming, it gives us a different perspective, auditing capabilities, and greater depth in our requirements.

HELLO WORLD

```
print("hello world")
```

Prints the string between parentheses on screen.

COMMENTS

To comment text, start the line with #.

```
# This is to explain the code
```

VARIABLES

```
counter=3
```

Assign 3 to the variable counter.

ADDITION AND MULTIPLICATION

```
counter=counter+1 # addition
```

```
counter=counter*2 # multiplication
```

SUBSTRINGS

```
text="hello world"  
  
print (text[2:6])
```

Prints the substring "llo w"

*In programming, counting usually starts at 0, so position 0 is the letter
h.*

KEYBOARD INPUT

Prints your name and stores it in the variable name.

```
name=input("What is your name? ")
```

MATHEMATICAL FUNCTIONS

```
from math import sin  
  
print(sin(45))
```

Prints the sine of 45 degrees.

GRAPHICS

To plot, it is necessary to install a package from the command line.

```
pip install matplotlib
```

And then we will create a text file called plot.py with this content

```
import matplotlib.pyplot as plt

x = [ 1 , 2 , 3 , 4 , 5 , 6 , 7 , 8 , 9 , 10 ]

y = [ 5 , 2 , 4 , 4 , 8 , 7 , 4 , 8 , 10 , 9 ]

plt.plot( x , y)
plt.xlabel( 'Time')
plt.ylabel('Units')
plt.show()
```

All that remains is to run it with `python plot.py`

CONDITIONALS

The following example examines the contents of variables `a` and `b`, determining whether `a` is greater than, less than, or equal to `b`.

```
a=5
b=8
if a>b:
    print("A is greater than b")
else:
    print("A is not greater than b")
if a==b:
    print("A equals b")
```

ARRAYS

```
myArray=[1,1.2,3.4]

myLength = len(myArray)

print(myArray[1])

print("Length: "+str(myLength))
```

FOR LOOP

```
motos = [ 'Ducati' , 'Gilera', 'Honda' ]  
  
for moto in motos:  
  
    print(moto)
```

WHILE

```
m = 10  
  
while m>5:  
    print(m)  
    m=m-1
```

FUNCTIONS

```
def suma (x,y):  
    return x + y  
  
print(suma(2,3))  
print(suma(1,1))
```

If more than one value needs to be returned, separate them with commas:

```
return suma, media
```

FILES

```
# write  
f= open ("file.txt" , "w" )  
f.write("Test")  
f.close()  
  
# read  
f= open ("file.txt" , "r" )  
data = f.read()  
print(data)  
f.close()
```

Writes a file named file.txt with the content "Test", then opens the file and prints its contents.

RANDOM

```
import random
print (random.randint(1,10))
```

WEB SCRAPING

```
import urllib.request

link = "https://ABCSite.com/?page=17"
f = urllib.request.urlopen(link)
myfile = f.read()
print(myfile)
```

INTERNET

ARPA (Advanced Research Projects Agency) emerged during the Cold War in response to the launch of the Soviet Sputnik satellite, bringing mathematicians, engineers, psychologists, and programmers together in its laboratories. In the late 1960s, ARPA sought a communications system resilient to failure, even if parts of the network were destroyed. Thus ARPANET was born, connecting four universities. Information on ARPANET was divided into independent fragments that traveled along variable routes and were reassembled at the destination. ARPANET evolved by adopting TCP/IP in 1983, and this was the origin of what is now known as the Internet.

TCP/IP

The Internet is based on the TCP/IP protocol suite. TCP (Transmission Control Protocol) ensures that packets arrive complete, in order, and without errors, while also managing flow control. IP (Internet Protocol) handles routing and addressing: each device has a unique address (such as 192.0.2.1) that indicates how data should be routed.

IP addresses include private ranges such as 10.0.0.0/8, 172.16.0.0/12, and 192.168.0.0/16 for internal networks.

The 127.0.0.0/8 block is for loopback: packets sent there return to the same machine (commonly 127.0.0.1 for local testing).

0.0.0.0 represents “all interfaces” or an undefined address, and 255.255.255.255 is used for broadcasts on the local network.

Subnet masks, such as 255.255.255.0, define which IP bits correspond to the network and which to the host. The “255” values (bits set to 1) mark the network portion, while zeros indicate hosts. This allows routers to quickly determine whether a packet should be routed locally or toward the Internet.

The exhaustion of IPv4 addresses drove the adoption of IPv6, with 128-bit addresses such as 2001:0db8::1.

DNS

Thanks to TCP/IP, any machine can communicate with another machine anywhere in the world. As the Internet grew, memorizing numerical IP addresses became impractical. DNS (Domain Name System) solved this problem with a distributed directory: when entering a domain such as “example.com”, a chain of servers translates it into its corresponding IP. Other technologies complete the ecosystem: IMAP, POP, and SMTP for email. HTTP/HTTPS to facilitate the request and delivery of web documents. When accessing a website, a sequential process begins: a DNS query to obtain the IP, sending a SYN packet for the TCP handshake, TLS negotiation for security, an HTTP/HTTPS request, and reception of bytes that the browser renders.

On October 29, 1969, the first message between UCLA and the Stanford Research Institute was attempted. Engineer Charley Kline wanted to transmit "LOGIN", but the system crashed after the letters "L" and "O". This "LO" became the first message on the Internet.

ARPA/DARPA also funded research into human-machine interaction, pattern recognition, language processing, planning, and systems capable of assisting decision-making. Its Information Processing Techniques Office, created in 1962, was particularly important to the early development of AI.

PRIVACY

"We don't spy... They call it telemetry"

Jean-Baptiste Kempf

Nerdearla Conference, Argentina 2026

Governments protect military and diplomatic communications; companies safeguard trade secrets, but individuals also need to preserve conversations, files, and data. It is possible to hide the existence of a message using invisible ink or steganography, change its representation, or transform its content so that it is unintelligible without a key.

ENCODING

Encoding consists of changing the representation of information so it can be stored or transmitted: ASCII converts characters into numbers and Base64 represents binary data as text. Because the rules are public and require no secret, encoding does not protect the content.

SUBSTITUTION

Replacing each letter, word, or symbol with another according to an agreed rule. In the Caesar cipher, for example, each letter is shifted by a fixed number of positions within the alphabet. The sentence is unreadable at a glance, but its protection is weak because it preserves regularities of the language, such as the frequency of certain letters and combinations. An adversary does not need to know the exact rule if they can reconstruct it by analyzing enough messages.

ENCRYPTION

For centuries, cryptography depended on shared keys in which sender and receiver had to know the same secret in advance. The problem is how to distribute the key: if it must be sent over the same channel being protected, it can be intercepted. Public-key cryptography, developed during the 1970s, proposed a radical solution: separate the key into two mathematically related parts. A public key can be distributed without restriction, while a private key remains under the control of its owner.

Prime numbers became especially important in some of these systems, particularly RSA, introduced by Ronald Rivest, Adi Shamir, and Leonard Adleman. To generate the keys, two large prime numbers are chosen and multiplied. Performing this multiplication is easy, but recovering the prime factors from the product is computationally expensive when the numbers are very large. The public key contains information derived from that product, while the private key depends on the original factors.

PGP

In 1991, American programmer Philip Zimmermann published PGP – Pretty

Good Privacy. His goal was to put strong cryptography within reach of ordinary people, especially to protect files and email against government, corporate, or criminal surveillance. Zimmermann understood privacy as a necessary condition for personal freedom and distributed the program free of charge.

The international circulation of PGP triggered a U.S. government investigation because, at the time, strong cryptographic software was subject to export restrictions similar to those applied to military material. Zimmermann responded creatively: in 1995 he published, through MIT Press, PGP Source Code and Internals, a huge book whose contents were PGP source code. Exporting that same code in digital form was restricted, but printed books could circulate internationally. Once outside the United States, the pages were scanned with OCR, the source code was reconstructed, and the program was compiled again. The investigation of Zimmermann was closed in January 1996 without charges.

PGP works as a hybrid cryptosystem. When a person encrypts a message, the program generates a random session key and uses a symmetric algorithm to encrypt the content. It then encrypts that small session key with the recipient's public key. When receiving the file, the recipient uses their private key to recover the session key and decrypt the message.

PGP also allows the creation of digital signatures. Instead of hiding the message, the signature produces a cryptographic fingerprint that is processed

with the author's private key. Anyone who possesses the author's public key can verify that the content was not altered and that the signature corresponds to that key. This is why PGP is said to provide confidentiality, integrity, and authenticity.

The complexity of PGP lies in key management: verifying who owns a public key, protecting the private key, keeping backups, and revoking a compromised key.

GPG

GPG (GNU Privacy Guard) is an implementation of OpenPGP. It allows files and messages to be encrypted, digitally signed, and public and private keys to be managed.

It is already installed in many Linux distributions. To check:

```
gpg --version
```

On Debian, Ubuntu, and derivatives it can be installed with:

```
sudo apt install gnupg
```

Create a key pair

```
gpg --full-generate-key
```

GPG asks for the key type and size, its validity period, and an identity normally composed of a name and email address. It also asks for a password to protect the private key.

List keys

```
gpg --list-keys
```

Export a public key

```
gpg --armor --export alguien@example.com > public-key.asc
```

The file `public-key.asc` can be sent to anyone who wants to encrypt information for us or verify our signatures.

Import someone else's public key

```
gpg --import public-key.asc
```

Sign a message

```
gpg --clearsign message.txt
```

This generates `message.txt.asc`, which contains the original readable text together with a digital signature. The signature allows verification that the content was not modified and that it was signed with the corresponding private key.

Verify the signature

```
gpg --verify message.txt.asc
```

Encrypt for a recipient

First, we need to have their public key imported. Then:

```
gpg --encrypt --armor --recipient bandini@example.com message.txt
```

The file is encrypted with the recipient's public key and can only be decrypted using their private key.

Decrypt

```
gpg --decrypt message.txt.asc
```

For example, if Juan wants to send Pedro a secret message, Juan needs Pedro's public key to encrypt it. Pedro then uses his private key to recover it. If Pedro also wants to prove that a message was really written by him, he can sign it with his private key, and anyone who has his public key can verify the signature.

STEGANOGRAPHY

Steganography consists of hiding information inside another file so that the existence of the message goes unnoticed. Unlike encryption, which makes a message unreadable but makes it evident that protected information exists, steganography attempts to hide the message itself. A classic technique consists of modifying the least significant bits of image pixels without producing changes perceptible to the human eye.

A simple example on Linux is using steghide to hide a text file inside a JPEG image:

```
# Install steghide
sudo apt install steghide

# Hide mensaje.txt inside foto.jpg
steghide embed -cf pic.jpg -ef msg.txt

# Extract the message
steghide extract -sf pic.jpg
```

DETECTING DEVICES THAT SPY ON US

The chapter on Maker Culture presents the principle of distrust toward devices that monitor us. This is not theoretical, but concrete. Eleven million Vizio E-Series and M-Series Smart TVs from 2014 to 2016 sampled screen pixels to generate a fingerprint and matched it against a database to determine what users were watching. Vizio combined this information with IP addresses, ZIP codes, and demographic information to sell to advertisers. Samsung, with its UE46ES8000 series, sent microphone recordings from TVs over port 443/TCP to vendors.

Personal assistants such as Amazon Echo recorded ambient conversations and sent them to Amazon S3 buckets. Google, for its part, sold Nest Secure guard systems with an undocumented microphone, and toys such as My Friend Cayla had Bluetooth and unsecured microphones that could be used to monitor the environment as well as to use the toy as a speaker.

To detect telemetry data and suspicious communications from our devices, we can carry out the following procedure.

Identify devices on the local network

```
sudo arp-scan --interface=wlan0 192.168.1.0/24
```

Observe traffic on 192.168.1.50 (example)

```
sudo tcpdump -i wlan0 host 192.168.1.50 -nn
```

Isolate DNS queries

```
sudo tcpdump -i wlan0 host 192.168.1.50 and port 53 -nn
```

Observe HTTPS/TLS connections

```
sudo tcpdump -i wlan0 host 192.168.1.50 and port 443 -nn
```

More detailed analysis

```
sudo tcpdump -i wlan0 host 192.168.1.50 -w device.pcap
```

BITCOIN

Bitcoin is a decentralized digital currency system proposed in 2008 by Satoshi Nakamoto. Its fundamental objective is to allow transfers of value between people without depending on banks, governments, or intermediaries that validate transactions. From one point of view, Bitcoin was not entirely new, but rather an intelligent combination of existing technologies: cryptography, peer-to-peer networks, and consensus mechanisms.

Speculation about Satoshi Nakamoto's identity has pointed to various figures in the world of cryptography and software, including Hal Finney, recipient of the first Bitcoin transaction; Nick Szabo, creator of Bit Gold; Adam Back, inventor of Hashcash; and Dorian Nakamoto, named in a journalistic investigation. Craig Wright was also identified, although his claims were widely challenged and he failed to prove that he controlled the cryptographic keys associated with Satoshi.

Other hypotheses hold that Nakamoto may have been a small group of developers rather than a single person. To this day, the real identity of Bitcoin's creator remains unknown.

Bitcoin transactions are recorded in a public, distributed database known as the blockchain. Instead of a central computer maintaining the ledger, multiple nodes keep independent copies and verify that transactions comply with the rules. This ledger makes it possible to mathematically demonstrate who has the ability to spend funds.

Control of the funds is carried out with public-key cryptography. Each user has a private key that allows them to digitally sign transactions. The network can verify that signature without knowing the private key. “Having” bitcoins means possessing the keys required to authorize their transfer. No physical asset needs to be stored – gold, paper – only a key.

The main problem with other cryptocurrencies used to be how to solve double spending, that is, how to prevent the same digital unit from being used simultaneously in two payments without resorting to a central authority. Bitcoin solves this with a mechanism called Proof of Work. Individuals, organizations, or companies have computers dedicated to mining. These computers compete using their computing power to build new transaction blocks. Retroactively modifying history is not viable because it means redoing that work and overtaking the rest of the network, something that becomes increasingly costly.

The protocol also establishes a predictable monetary policy. As a reward for securing the network, new bitcoins are issued, but that issuance decreases every four years through a process called halving. The maximum amount is limited to 21 million bitcoins.

Ultimately, Bitcoin replaces institutional trust with mathematical rules, economic incentives, and distributed consensus.

FINAL NOTES

The third edition reorganizes the theoretical sections, expands the content, and adds reference chapters on privacy and cryptocurrencies.

I want to especially thank the students who enrolled in the Maker Counterculture workshops and the attendees of the talks. These spaces were the starting point for turning my notes into the book's introductory material. Thanks also to everyone who took a moment to provide feedback or a reference.

Roni Bandini

Buenos Aires, Argentina

TABLE OF CONTENTS

Introduction	1
Machines, automata, robots and cyborgs	3
Maker Culture	11
Maker Counterculture	16
Artificial Intelligence	30
Electronics	44
Arduino	54
UART	63
LORA	67
Binary System	71
Linux	75
Programming	81
Internet	91
Privacy	95
Bitcoin	105
Final Notes	108
Table of Contents	110
Bibliography	112

BIBLIOGRAPHY AND REFERENCES

Taylor, F. W.. The Principles of Scientific Management. Harper & Brothers, 1911. ISBN: 978-0486299889.

Shannon, C. E.. A Mathematical Theory of Communication. Bell System Technical Journal, 1948. DOI: 10.1002/j.1538-7305.1948.tb01338.x.

Russell, S.; Norvig, P.. Artificial Intelligence: A Modern Approach. Pearson, 2020. ISBN: 978-0134610993.

Situnayake, D.; Plunkett, J.. AI on the Edge. O'Reilly Media, 2022. ISBN: 978-1098120207.

Domingos, P.. The Master Algorithm. Basic Books, 2015. ISBN: 978-0465065707.

Platt, C.. Make: Electronics. Maker Media, 2015. ISBN: 978-1680450262.

Muslin, E.. Máquinas del siglo XX. Editorial MIR, 1974.

Standage, T.. The Mechanical Turk. Walker & Company, 2002. ISBN: 978-0802714309.

Posselt, E. A.. The Jacquard Machine Analyzed and Explained. John Wiley & Sons, 1902.

Editora Verde Olivo. Con nuestros propios esfuerzos. Editora Verde Olivo, 1992.

Shelley, M.. *Frankenstein; or, The Modern Prometheus*. Lackington, Hughes, Harding, Mavor & Jones, 1818.

Dick, P. K.. *Do Androids Dream of Electric Sheep?*. Doubleday, 1968. ISBN: 978-0345404473.

Dahl, R.. *The Sound Machine*. The New Yorker, 1949.

Roszak, T.. *The Making of a Counter Culture*. Doubleday, 1969. ISBN: 978-0520212270.

Gibson, W.. *Neuromancer*. Ace Books, 1984. ISBN: 978-0441569595.

Baudrillard, J.. *Simulacra and Simulation*. University of Michigan Press, 1994. ISBN: 978-0472065219.

Ashby, W. R.. *An Introduction to Cybernetics*. Chapman & Hall, 1956. ISBN: 978-0416683009.

Alonso, R.. *Elogio de la low-tech*. Fundación Telefónica / catalog, 2015.

Simondon, G.. *El modo de existencia de los objetos técnicos*. Aubier, 1958.

ISBN: 978-2700700573.

Crawford, M. B.. *Shop Class as Soulcraft*. Penguin Press, 2009. ISBN: 978-0143117469.

Mumford, L.. *Technics and Civilization*. Harcourt, Brace and Company, 1934. ISBN: 978-0226550275.

Burroughs, W. S.. *Naked Lunch*. Olympia Press, 1959. ISBN: 978-0802132958.

Clynes, M. E.; Kline, N. S.. *Cyborgs and Space*. *Astronautics*, 1960. Article reference.

Čapek, K.. *R. U. R. (Rossum's Universal Robots)*. Aventinum, 1920.

Weizenbaum, J.. *ELIZA – A Computer Program for the Study of Natural Language Communication*. *Communications of the ACM*, 1966. DOI: 10.1145/365153.365168.

Vaswani, A.; Shazeer, N.; Parmar, N.; Uszkoreit, J.; Jones, L.; Gomez, A. N.; Kaiser, Ł.; Polosukhin, I.. *Attention Is All You Need*. *NeurIPS*, 2017. arXiv:1706.03762.

McCarthy, J.; Minsky, M. L.; Rochester, N.; Shannon, C. E.. *A Proposal for the Dartmouth Summer Research Project on Artificial Intelligence*. Dartmouth

College, 1955. Historical document.

Rosenblatt, F.. The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain. *Psychological Review*, 1958. DOI: 10.1037/h0042519.

Rumelhart, D. E.; Hinton, G. E.; Williams, R. J.. Learning representations by back-propagating errors. *Nature*, 1986. DOI: 10.1038/323533a0.

Postman, N.. *Technopoly: The Surrender of Culture to Technology*. Alfred A. Knopf, 1992. ISBN: 978-0679745402.

Banzi, M.; Shiloh, M.. *Getting Started with Arduino*. Maker Media, 2022. ISBN: 978-1457186439.

Thompson, H. S.. *The Great Gatsby* (copying exercise described in Gonzo practice). F. Scott Fitzgerald / Charles Scribner's Sons, 1925.

Maker Counterculture

Version 3

Roni Bandini

Buenos Aires, Argentina

October 2026

CC BY 4.0 License

The contents of this book are provided for educational, experimental, and documentary purposes. The author does not guarantee that the information, procedures, code, circuits, designs, or examples are free of errors or suitable for any particular purpose. The practical application of the contents and its possible consequences are the responsibility of the person using them. Readers are encouraged to verify the information and take the appropriate safety measures and precautions before carrying out any implementation.